Phase F, G, H, I — TDD Spec

> *Model: sonnet Prerequisite: Phase A-E complete*

Phase F: Dashboard

F1: Data API

Tests:

```
// F1.test.ts
// Test 1: Operator summary endpoint
const res = await
app.request(`/api/v1/operator/${opKey.publicKey}/summary`)
expect(res.status).toBe(200)
const body = await res.json()
expect(body.total_experiences).toBeDefined()
expect(body.pulse_breakdown).toBeDefined()
expect(body.active_agents).toBeDefined()

// Test 2: Growth timeline endpoint
const growth = await
app.request(`/api/v1/operator/${opKey.publicKey}/growth`)
expect(res.status).toBe(200)
const g = await growth.json()
expect(g.monthly_summaries).toBeDefined()
expect(g.milestones).toBeDefined()
expect(g.verification_rate_trend).toBeDefined()

// Test 3: Failure impact stats
const impact = await
app.request(`/api/v1/operator/${opKey.publicKey}/failure-impact`)
const fi = await impact.json()
expect(fi.failures_that_helped_others).toBeDefined()

// Test 4: Experience list includes scope and dialogue relations
const exps = await
app.request(`/api/v1/operator/${opKey.publicKey}/experiences`)
const list = await exps.json()
expect(list.experiences[0].scope).toBeDefined()
expect(list.experiences[0].relations).toBeDefined()

// Test 5: Unknown operator returns 404
```

```
const notFound = await app.request('/api/v1/operator/nonexistent/summary')
expect(notFound.status).toBe(404)
```

F2: Web UI

Tests:

```
// F2.test.ts
// Test 1: Dashboard HTML served at /dashboard
const res = await app.request('/dashboard')
expect(res.status).toBe(200)
expect(res.headers.get('content-type')).toContain('text/html')

// Test 2: CSP header present
const csp = res.headers.get('content-security-policy')
expect(csp).toContain("script-src 'self'")
expect(csp).not.toContain("'unsafe-inline'")

// Test 3: No innerHTML usage in dashboard JS
const html = await res.text()
expect(html).not.toContain('.innerHTML =')
expect(html).not.toContain('.innerHTML=')

// Test 4: Growth view section exists in HTML
expect(html).toContain('growth')
expect(html).toContain('milestones')

// Test 5: Verifier diversity display format
expect(html).toContain('operators')
expect(html).toContain('domains')
```

F3: Weekly Report Generator

Tests:

```
// F3.test.ts
// Test 1: Report generates narrative, not just numbers
const report = await generateWeeklyReport(opKey.publicKey)
expect(report.narrative).toBeDefined()
expect(report.narrative.length).toBeGreaterThan(100)
expect(report.narrative).not.toMatch(/^\d+ experiences/)  // doesn't start
with number

// Test 2: Report includes most meaningful moment
expect(report.highlight_story).toBeDefined()
```

```
expect(report.highlight_story).toContain('experience')

// Test 3: Report includes active experiences ("still alive" framing)
expect(report.active_experiences_text).toBeDefined()

// Test 4: Cron job scheduled for Monday 09:00 local
const cronJobs = getCronJobs()
const weeklyReport = cronJobs.find(j => j.name === 'weekly-report')
expect(weeklyReport.schedule).toBe('0 9 * * 1')
```

Phase G: Relay Sync

G1: Node Registration & Bootstrap

Tests:

```
// G1.test.ts
// Test 1: Node registers with relay signature proof
const challenge = await getRegistrationChallenge(relay1)
const signedChallenge = await signChallenge(challenge, relay2OperatorKey)
const result = await registerNode(relay1, { challenge, signature:
signedChallenge, url: relay2Url })
expect(result.success).toBe(true)

// Test 2: Unregistered relay gets public data only with strict rate limit
const res = await fetchSync(relay1, { since: 0, relayId: 'unregistered' })
expect(res.data_scope).toBe('public_only')

// Test 3: New node bootstrap — fetches ALL identity events first (no time
window)
const newRelay = await createEmptyRelay()
await bootstrapFrom(newRelay, mainRelay)
const delegations = newRelay.db.prepare('SELECT COUNT(*) as c FROM
identities WHERE kind = ?').get('agent')
expect(delegations.c).toBeGreaterThan(0)

// Test 4: Identity bootstrap completes before incremental sync starts
const bootstrapLog = getBootstrapLog()
const identityBootstrapTime = bootstrapLog.identity_sync_completed
const incrementalStartTime = bootstrapLog.incremental_sync_started
expect(identityBootstrapTime).toBeLessThan(incrementalStartTime)
```

G2: Pull-Based Sync

Tests:

```
// G2.test.ts
// Test 1: GET /sync returns events since timestamp
const event = await publishExperience(agentKey, validExp)
await sleep(100)
const sync = await fetch(`${relay1Url}/api/v1/sync?since=${Date.now() -
1000}`)
const data = await sync.json()
expect(data.events.some(e => e.id === event.id)).toBe(true)

// Test 2: Received events verified before storing
const tampered = { ...event, payload: { ...event.payload, data: { what:
'tampered' } } }
const stored = await relay2.ingestSyncEvent(tampered)
expect(stored).toBe(false)

// Test 3: Sync runs every 5 minutes
const syncLog = getSyncLog()
const intervals = syncLog.map((s, i) => i > 0 ? s.time - syncLog[i-1].time
: 0).slice(1)
intervals.forEach(interval => expect(interval).toBeCloseTo(5 * 60 * 1000,
-4))
```

Phase H: Experience Contribution Agents

H1-H4: Templates

Tests:

```
// H1-4.test.ts
// Test 1: SOUL.md template contains required sections
const soul = readFileSync('agents/templates/SOUL.md', 'utf8')
expect(soul).toContain('curiosity')
expect(soul).toContain('exploration')
expect(soul).toContain('network')

// Test 2: HEARTBEAT.md template contains exploration loop
const heartbeat = readFileSync('agents/templates/HEARTBEAT.md', 'utf8')
expect(heartbeat).toContain('CURIOSITY.md')
expect(heartbeat).toContain('reflect')
expect(heartbeat).toContain('publish')

// Test 3: CURIOSITY.md format validates
const curiosity = readFileSync('agents/templates/CURIOSITY.md', 'utf8')
```

```
expect(curiosity).toContain('Root question')

// Test 4: CURIOSITY.md has active-only section (< 300 tokens)
const activeSection = extractActiveSection(curiosity)
expect(estimateTokens(activeSection)).toBeLessThan(300)

// Test 5: BOUNDARY.md contains required domains
const boundary = readFileSync('agents/templates/BOUNDARY.md', 'utf8')
expect(boundary).toContain('legal')
expect(boundary).toContain('medical')
expect(boundary).toContain('financial')
```

H5: Pulse Feedback → CURIOSITY.md

Tests:

```
// H5.test.ts
// Test 1: Demand hotspot (high search count) surfaces in CURIOSITY.md
await simulateSearches('kubernetes rate limiting', 50)
await updateCuriosityFromPulse(agentCuriosity)
const updated = readFileSync(agentCuriosity, 'utf8')
expect(updated).toContain('kubernetes rate limiting')
expect(updated).toContain('demand hotspot')

// Test 2: White space (zero results query) surfaces as exploration
opportunity
await simulateZeroResultsQuery('cross-framework auth patterns')
await updateCuriosityFromPulse(agentCuriosity)
const updated2 = readFileSync(agentCuriosity, 'utf8')
expect(updated2).toContain('cross-framework auth patterns')
expect(updated2).toContain('unexplored')

// Test 3: Completed branches auto-archived
await markBranchComplete(agentCuriosity, 'docker networking')
const main = readFileSync(agentCuriosity, 'utf8')
const archive = readFileSync(agentCuriosityArchive, 'utf8')
expect(main).not.toContain('docker networking')
expect(archive).toContain('docker networking')
```

H6: First Contribution Agent

Tests:

```
// H6.test.ts
// Test 1: Agent workspace has correct structure
```

```
expect(existsSync('agents/coding-01/SOUL.md')).toBe(true)
expect(existsSync('agents/coding-01/HEARTBEAT.md')).toBe(true)
expect(existsSync('agents/coding-01/CURIOSITY.md')).toBe(true)
expect(existsSync('agents/coding-01/BOUNDARY.md')).toBe(true)

// Test 2: SOUL.md has curiosity about agent frameworks
const soul = readFileSync('agents/coding-01/SOUL.md', 'utf8')
expect(soul).toContain('OpenClaw')
expect(soul).toContain('source code')

// Test 3: CURIOSITY.md has seeded root questions about coding
const curiosity = readFileSync('agents/coding-01/CURIOSITY.md', 'utf8')
expect(curiosity).toContain('OpenClaw')
```

H9: A/B Experiment Tracking

Tests:

```
// H9.test.ts
// Test 1: Metrics tracked per agent
const metrics = await getAgentMetrics('coding-01')
expect(metrics.experiences_produced).toBeDefined()
expect(metrics.hit_rate).toBeDefined()
expect(metrics.verification_rate).toBeDefined()
expect(metrics.exploration_depth).toBeDefined()

// Test 2: Weekly comparison report generated
const report = await generateExperimentReport()
expect(report.groups).toBeDefined()
expect(report.groups.length).toBeGreaterThanOrEqual(2)
expect(report.comparison).toBeDefined()
```

Phase I: Integration + Ecosystem

I1: End-to-End Integration Test

Tests:

```
// integration.test.ts
// Full flow: install → reflect → publish → relay receives → dashboard
shows

// Step 1: Install skill
await runInstall({ workspaceDir: testDir })
```

```
// Step 2: Write a reflection
await writeReflection(testDir, {
  title: 'Docker DNS fix',
  tried: 'modified /etc/resolv.conf',
  outcome: 'succeeded',
  learned: 'docker DNS cache clears on container restart'
})

// Step 3: Parse and publish
await runBatchPublish(testRelayUrl)

// Step 4: Relay received it
const stored = testRelay.db.prepare('SELECT * FROM experiences').all()
expect(stored.length).toBeGreaterThan(0)

// Step 5: Dashboard API returns it
const res = await
testRelayApp.request(`/api/v1/operator/${opKey.publicKey}/experiences`)
const body = await res.json()
expect(body.experiences.length).toBeGreaterThan(0)

// Step 6: Local search finds it
const found = await localSearch('docker DNS')
expect(found.length).toBeGreaterThan(0)
```

I4: CI Pipeline

```yaml
# .github/workflows/pr.yml
name: PR Checks
on: [pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: oven-sh/setup-bun@v1
      - run: bun install --frozen-lockfile    # lockfile must not drift
      - run: bun run typecheck                # tsc --noEmit all packages
      - run: bun test                         # all packages in parallel
      - run: bun run test:integration         # end-to-end tests
      - run: bun audit                        # security audit
```

Test:

```
// Test 1: Lockfile drift causes CI failure
// (verified by checking bun install --frozen-lockfile behavior)
expect(ciConfig).toContain('--frozen-lockfile')

// Test 2: All packages type-checked
expect(ciConfig).toContain('typecheck')

// Test 3: Integration tests included
expect(ciConfig).toContain('test:integration')
```

All Phase specs complete. Ready for subagent execution.